VOL. 23

Swift

From variables and optionals to actors, SwiftUI views and SwiftPM packages

Source	docs.swift.org (fallback, swift.org/learn moved)
Generated	2026-10-10
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 23 - Swift

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 6
Chapter map 7
Chapter 1 - Swift tour: setup and first look 7
1.1 Summary 7
1.2 Concepts 8
What the toolchain contains 8
Three ways to write Swift 8
Anatomy of a first program 9
`print` and string interpolation 9
Type inference 9
Style conventions 9
1.3 Examples 10
1.4 Common mistakes and best practices 12
1.5 Practice exercises 12
Solutions 13
1.6 Mini test 13
Answer key and explanations 13
Chapter 2 - Variables, constants and basic types 14
2.1 Summary 14
2.2 Concepts 14
Declaring values 14
The numeric types 15
Explicit conversion 15
Booleans and conditions 15
Strings and characters 15
Tuples 16
`typealias` and `Any` 16
2.3 Examples 16
2.4 Common mistakes and best practices 19
2.5 Practice exercises 20
Solutions 20
2.6 Mini test 21
Answer key and explanations 21

Chapter 3 - Optionals . . . 21
3.1 Summary . . . 21
3.2 Concepts . . . 22
What an optional is . . . 22
`Optional` is an enumeration . . . 22
Choosing an unwrapping strategy . . . 22
Nil-coalescing . . . 23
Optional chaining . . . 23
Implicitly unwrapped optionals . . . 23
Sources of optionals . . . 23
3.3 Examples . . . 24
3.4 Common mistakes and best practices . . . 27
3.5 Practice exercises . . . 28
Solutions . . . 28
3.6 Mini test . . . 29
Answer key and explanations . . . 29
Chapter 4 - Operators and expressions . . . 30
4.1 Summary . . . 30
4.2 Concepts . . . 30
The operator groups . . . 30
Integer arithmetic . . . 31
Ranges . . . 32
Bitwise operators . . . 32
Short-circuiting . . . 32
Precedence and custom operators . . . 32
4.3 Examples . . . 33
4.4 Common mistakes and best practices . . . 36
4.5 Practice exercises . . . 37
Solutions . . . 37
4.6 Mini test . . . 37
Answer key and explanations . . . 38
Chapter 5 - Control flow and early exits . . . 38
5.1 Summary . . . 38
5.2 Concepts . . . 39
The conditional statements . . . 39

`switch` rules . . . 39
The pattern forms . . . 40
Loops . . . 40
`defer` . . . 40
5.3 Examples . . . 40
5.4 Common mistakes and best practices . . . 45
5.5 Practice exercises . . . 46
Solutions . . . 46
5.6 Mini test . . . 47
Answer key and explanations . . . 48
Chapter 6 - Functions and closures . . . 48
6.1 Summary . . . 48
6.2 Concepts . . . 49
Argument labels and parameter names . . . 49
A summary of parameter forms . . . 49
Return values . . . 49
Function types . . . 49
Closures . . . 50
Capture and escape . . . 50
Higher-order functions . . . 50
6.3 Examples . . . 51
6.4 Common mistakes and best practices . . . 55
6.5 Practice exercises . . . 56
Solutions . . . 56
6.6 Mini test . . . 57
Answer key and explanations . . . 57
Chapter 7 - Structures, classes and enumerations . . . 58
7.1 Summary . . . 58
7.2 Concepts . . . 58
Value types versus reference types . . . 58
Properties . . . 59
Methods and subscripts . . . 59
Inheritance . . . 60
Initialisation . . . 60
Automatic reference counting . . . 60

Choosing a type 61
7.3 Examples 61
7.4 Common mistakes and best practices 68
7.5 Practice exercises 69
Solutions 69
7.6 Mini test 70
Answer key and explanations 71

How to use this volume

Prerequisites. Any previous programming language. You should be comfortable with variables, functions, loops, arrays and the idea that a compiler turns source text into a program. No Apple platform experience and no Xcode license is required: every runnable example in this volume is a console program that works on macOS, Linux and WSL. If you have read Vol. 03 (JavaScript), Vol. 09 (Python) or Vol. 10 (Java) you have enough background.

Estimated study time. About 25 hours. Chapters 1 to 6 build the language core, chapters 7 to 11 cover the type system, runtime and safety topics that make Swift different, and chapters 12 and 13 add the app and package layers.

How the chapters work. Each chapter opens with a summary, explains the concepts with tables and small diagrams, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and closes with practice exercises, their solutions and a mini test with an answer key.

How to run the code. Save a program as `main.swift` and run it with `swift main.swift`. The `swift` driver compiles the file and starts the resulting executable, so a single command is enough for one-file programs. Blocks tagged `swift` are complete programs; blocks tagged `text` show what the program prints; blocks tagged `bash` are terminal commands. Each `swift` block is a standalone program: copy it into a file, add the parts you need, and run it. The SwiftUI chapter (12) cannot run in a console, so its code blocks are labelled as not executed.

shell

```
swift --version
swift main.swift
```

output

```
Swift version 6.4 (swift-6.4-RELEASE)
Target: x86_64-unknown-linux-gnu
Hello, Swift!
```

Version numbers and target triples differ on every machine; the shape of the output does not.

A note about verification. Swift is not installed on the machine used to write this volume, so no example in this book was executed. Every program was written against the documented behaviour of the Swift standard library and reviewed line by line, and every output block is the expected output described by the language documentation. Where a result depends on the operating system, the compiler version or on hash ordering, the text says so instead of printing a made-up value. Install a toolchain and compare the outputs with your own runs; that exercise is part of the learning.

About the source. The syllabus follows the official Swift documentation published at docs.swift.org, which hosts The Swift Programming Language (TSPL) and the Swift Package Manager documentation. The `swift.org/learn` index page no longer resolves, so the book chapters listed in `# Sources` are used as the fallback curriculum. Only topic names are taken from the source: all explanations, examples and exercises in this volume are original writing,

and the sample code was written for this book.

Chapter map

#	Chapter	You will learn
1	Swift tour: setup and first look	the toolchain, scripts and packages, your first program, `print`
2	Variables, constants and basic types	`let` and `var`, numeric types, `Bool`, strings, tuples, `typealias`
3	Optionals	`nil`, `if let`, `guard let`, nil-coalescing, optional chaining
4	Operators and expressions	arithmetic, comparison, logical, ranges, bitwise, overflow operators
5	Control flow	`if`, `switch`, loops, `defer`, `guard` and early exit
6	Functions and closures	argument labels, defaults, variadic, `inout`, function types, closures
7	Structures, classes and enumerations	value and reference semantics, properties, methods, subscripts, inheritance, initialisers
8	Collections	arrays, sets, dictionaries, copy-on-write, higher-order functions
9	Protocols and generics	requirements, delegation, associated types, type parameters, `some` and `any`
10	Error handling and `Result`	throwing functions, `do`/`catch`, `try?`, `Result`
11	Concurrency	`async`/`await`, structured concurrency, task groups, actors
12	SwiftUI: your first views	declarative views, modifiers, state, lists, previews
13	Tooling and packages	SwiftPM layout, `swift build` and `swift test`, XCTest, formatting
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Swift tour: setup and first look

1.1 Summary

- Swift is a compiled, statically typed language maintained as an open source project by the Swift community and Apple.
- The toolchain ships a compiler (`swiftc`), a driver with build, run and test subcommands (`swift`), a debugger (LLDB) and the Swift Package Manager (SwiftPM).

- A one-file program stores top-level statements in a file, typically named `main.swift`, and runs with `swift main.swift`.
- Larger programs become SwiftPM packages with a `Package.swift` manifest, a `Sources` directory and a `Tests` directory.
- `print` writes its arguments followed by a newline to standard output.
- Values have types, and the compiler infers most of them from the literals and expressions you write.
- `let` introduces a constant that cannot be changed, while `var` introduces a variable that can.
- String interpolation lets any expression be embedded inside a string literal with `\(expression)`.
- Type checking is strict and happens before the program runs, so most mistakes appear as compile-time errors rather than runtime crashes.
- The standard library provides the numeric, string and collection types plus the concurrency primitives; platform features such as networking live in extra modules.

1.2 Concepts

What the toolchain contains

Command or tool	What it does
`swiftc`	compiles one or more `.swift` files into an executable
`swift file.swift`	compiles and immediately runs a single file
`swift build`	builds the package in the current directory
`swift run`	builds, then runs the package's executable product
`swift test`	builds and runs the test bundle
`swift repl`	starts the interactive read-eval-print loop
`lldb`	the debugger used for breakpoints and stack inspection
`swift-format`	opinioned code formatter, also available as a SwiftPM plugin
SourceKit-LSP	the language server that gives editors completion and jump-to-definition

Swift is available for macOS, Linux and Windows. Windows and Linux toolchains are distributed from the download page on swift.org; the Linux and WSL route is usually the smoothest one for the console programs in this volume.

Three ways to write Swift

output

```
  one file            package                       application
+-------------+     +-------------------+        +------------------+
| main.swift  |     | Package.swift     |        | Xcode project or |
| swift x.swift|    | Sources/...       |        | SwiftPM bundle   |
+-------------+     | Tests/...         |        | UI framework     |
  learning,         | swift build/run   |        | (AppKit, SwiftUI)|
  scripts           +-------------------+        +------------------+
```

Everything in chapters 1 to 11 and chapter 13 uses the first two shapes. Chapter 12 shows a third shape: a view tree built by a UI framework.

Anatomy of a first program

output

```
main.swift
+---------------------------------------------+
| constants and variables                     |
| functions, types and extensions              |
| top-level statements, executed in order      |
+---------------------------------------------+
        |
        v  swift main.swift
   compile -> link -> run -> print to stdout
```

Top-level statements execute top to bottom, in the order they appear. In a package the entry point is the file `main.swift` inside the executable target's directory; a library package has no `main.swift` and is not runnable on its own.

`print` and string interpolation

`print` accepts any number of items and three optional parameters: `separator` (default a single space), `terminator` (default a newline) and `to` (a target such as standard error). The simplest form prints one value per call.

String interpolation is written as `\\(expression)` inside a string literal. The expression can be a variable, an arithmetic result, a function call or a computed property, and it may itself contain a string literal.

Type inference

Swift reads the literal on the right-hand side and gives the variable or constant a type. Inference is not guesswork: `42` is an `Int`, `4.2` is a `Double`, `"Hi"` is a `String`, and `[1, 2]` is an `[Int]`. You can inspect the inferred type with `type(of:)`, which is useful while learning and while reading someone else's code.

Style conventions

Element	Convention
Type, protocol, enum names	`UpperCamelCase` (`BankAccount`, `Downloadable`)
Variables, functions, properties, enum cases	`lowerCamelCase` (`accountNumber`, `loadPage()`)
Semicolons	optional, one statement per line

Element	Convention
Braces	always on the same line as the declaration that opens them
Line length	a comfortable 80 to 100 columns

1.3 Examples

Example 1: the smallest program that prints.

Swift

```swift
print("Hello, Swift!")
```

output

```
Hello, Swift!
```

Example 2: interpolation, escapes and multiple arguments.

Swift

```swift
let language = "Swift"
let version = 6.4
print("Learning \\(language) \\(version)")
print("line one\nline two")
print("tabbed\tvalue")
print("quote: \"keep\" backslash: \\\\")
print("scalar:", "U+1F600", "\u{1F600}")
```

output

```
Learning Swift 6.4
line one
line two
tabbed	value
quote: "keep" backslash: \
scalar: U+1F600 😀
```

The `\n` inside the first literal becomes a real newline, and the escaped backslash `\\` prints as one backslash.

Example 3: several values, a custom separator and a custom terminator.

Swift

```swift
print("Ada", "Linus", "Grace")
print("Ada", "Linus", separator: " and ")
print("no newline at the end", terminator: ".")
print()
```

output

```
Ada Linus Grace
Ada and Linus
no newline at the end.
```

`print()` with no arguments prints just the newline that the previous call omitted.

Example 4: comments.

Swift

```swift
// A line comment runs to the end of the line.
let limit = 10          // A comment may follow code on the same line.

/* Block comments can span
   several lines. */

/* They may also be /* nested */ inside another block comment. */

print(limit)
```

output

```
10
```

Example 5: constants, variables and inference.

Swift

```swift
let maximumSpeed = 120
var currentSpeed = 80
print(type(of: maximumSpeed), type(of: currentSpeed))
currentSpeed = 95
print("Now going \\(currentSpeed) of \\(maximumSpeed)")
print("Freeway: \\(maximumSpeed == 120)")
```

output

```
Int Int
Now going 95 of 120
Freeway: true
```

`maximumSpeed` is a constant, so a second assignment to it is a compile-time error; `currentSpeed` is a variable and can be reassigned at any time.

Example 6: the command line.

shell

```shell
swift --version
```

output

```
Swift version 6.4 (swift-6.4-RELEASE)
Target: x86_64-unknown-linux-gnu
```

shell

```shell
swift main.swift
```

output

```
Hello, Swift!
```

Example 7: the interactive loop.

shell

```
swift repl
```

output

```
Welcome to Swift version 6.4.
Type :help for assistance.
  1> let name = "Swift"
name: String = "Swift"
  2> print("Hi, \\(name)")
Hi, Swift
  3> :quit
```

The loop keeps the state of previous lines, which makes it a good place to test an expression you are unsure about before putting it in a file.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing top-level statements in a non-entry file of a package	the compiler reports that expressions are not allowed at the top level	keep statements in `main.swift`, or in a function called from it
Expecting Swift to be interpreted	errors appear at compile time, not when the line runs	run `swift main.swift` and read the compiler message
Using `var` for values that never change	the reader cannot tell what may change	default to `let` and only use `var` when you assign again
Naming types in `lowerCamelCase`	breaks the ecosystem convention and reads oddly next to types from the standard library	types in `UpperCamelCase`, members in `lowerCamelCase`
Printing debug data with `print` and leaving it in	production logs fill up and slow the program	use a dedicated logging facility, or remove the calls
Assuming `print` writes to the error stream	messages disappear in normal output	use `print(..., to: .standardError)` or a logger for errors

- Read the first compiler error first: later errors are often caused by the first one, and Swift reports the exact line and column.
- Keep one file runnable while learning; the package layout from chapter 13 pays off as soon as a project has more than a few hundred lines.

1.5 Practice exercises

1. Which command runs a single Swift file, and which command builds the package in the current directory?
2. Write a one-line program that prints `Swift 6.4 is compiled, not interpreted`. Use constants for the numbers and words so that the line uses interpolation.
3. What is the type of each value: `7`, `7.0`, `true`, `"7"`, `[7]`?

4. Explain why `let x = 2` followed by `x = 3` fails, and what the fix is.
5. Which `print` call produces `a,b,c` and then no newline at all?

Solutions

1. `swift main.swift` runs a single file; `swift build` builds the package in the current directory (`swift run` also runs it).
2. One solution:

Swift

```swift
let version = 6.4
print("Swift \\(version) is compiled, not interpreted")
```

output

```
Swift 6.4 is compiled, not interpreted
```

3. `Int`, `Double`, `Bool`, `String`, `[Int]`.
4. `let` introduces a constant; assigning again is rejected by the compiler because the value can never change, which is what makes `let` safe to share. Use `var x = 2` when the value must change.
5. `print("a", "b", "c", separator: ",")` prints the items separated by commas and ends with a newline. To avoid the final newline, add `terminator: ""`.

1.6 Mini test

1. What does `swift main.swift` do? A) Starts the interactive loop B) Compiles the file and runs the resulting program C) Only checks the syntax D) Installs the dependencies listed in `Package.swift`
2. Which symbol starts a line comment in Swift? A) `#` B) `//` C) `--` D) `/*`
3. What does `\\(2 + 3)` produce inside a string literal? A) The text `\\(2 + 3)` B) `5` C) `2 + 3` D) A compile error
4. True or false: a variable declared with `var` may be declared with a type that never changes and still be reassigned.
5. Which separator does `print` use when none is given? A) A comma B) Nothing at all C) A single space D) A semicolon
6. Which statement about Swift's type checking is true? A) Types are checked while the program runs B) Types are checked by the compiler before the program runs C) Swift has no type checking because it infers types D) Only function results are checked
7. Where does a package's executable entry point live? A) `Package.swift` B) `Tests/main.swift` C) `Sources/<target>/main.swift` D) `Sources/main.swift`
8. What is the printed result of `print("one", terminator: "")` followed by `print("two")`?

Answer key and explanations

1. B - the driver compiles the file, links it and runs the result in one command.

2. B - `//` comments to the end of the line; `/* */` is the block form.
3. B - interpolation evaluates the expression, so the string contains the value `5`.
4. True - `var` allows reassignment; the type of a variable is fixed, but its value is not.
5. C - the default separator is one space and the default terminator is a newline.
6. B - Swift is statically typed, so most errors are caught at compile time, not at runtime.
7. C - the executable target is a directory under `Sources`, and the entry file inside it is `main.swift`.
8. `onetwo` - the first call prints without a newline, so the second call continues on the same line.

Chapter 2 - Variables, constants and basic types

2.1 Summary

- `let` declares a constant and `var` declares a variable; the choice documents intent and enables compiler optimisations.
- A type annotation is written as `name: Type` and is only needed when inference would pick the wrong type.
- `Int` is the platform's default integer type, with fixed-size relatives such as `Int8`, `UInt32` and `Int64`.
- `Double` is the default floating point type; `Float` is the 32-bit variant and is rarely used explicitly.
- `Bool` holds `true` or `false`, and Swift refuses to treat any other value as a condition.
- `String` is a collection of `Character` values, is value typed, and supports interpolation, concatenation and a large property and method API.
- Tuples group a fixed number of values of any types, with optional names for their elements.
- `typealias` gives a new name to an existing type, which helps readability at call sites.
- `Any` can hold a value of any type, but using it throws away the type safety that motivates Swift.
- Conversions between numeric types are explicit, because `Int` and `Double` are different types and nothing is converted silently.

2.2 Concepts

Declaring values

output

```
let name: Type = value        constant, cannot be reassigned
var name: Type = value        variable, may be reassigned
let name = value              the type is inferred
```

The annotation and the initial value may not both be missing: a declaration either has a type or a value the compiler can use to infer one.

The numeric types

Family	Types	Notes
Signed integers	`Int`, `Int8`, `Int16`, `Int32`, `Int64`	`Int` matches the platform word size
Unsigned integers	`UInt`, `UInt8`, `UInt16`, `UInt32`, `UInt64`	used for bit patterns and low-level code
Floating point	`Double`, `Float`	`Double` has about 15 decimal digits of precision
Decimal	`Decimal`	base-10 arithmetic, from Foundation, for money

Integer literals can be written with _ separators, in binary, octal or hexadecimal form, and floating point literals accept a decimal exponent or a hexadecimal exponent. The compiler infers `Int` for a whole number and `Double` for a number with a decimal point or an exponent.

Explicit conversion

Nothing is converted implicitly between numeric types, so converting is always a call. The conversions are initialisers:

output

```
Int(3.9)          -> 3          truncates toward zero
Int(-3.9)         -> -3         truncates toward zero
Double(3)         -> 3.0
Int("42")         -> Optional(42)      nil when the text is not a number
UInt8(300)        -> runtime error     out of range for the type
```

Converting a text value to a number fails when the text is not a valid number, which is why the result is an optional (see chapter 3).

Booleans and conditions

A condition must be a `Bool`. Numbers are not conditions, so `if 1 { }` is a compile error, and so is an optional used directly as a condition. Swift requires an explicit comparison such as `if count > 0` or an explicit unwrap such as `if let name`.

Strings and characters

Operation	Example
Concatenation	`"ab" + "cd"` gives `"abcd"`
Interpolation	`"count: \\(messages.count)"`

Operation	Example
Length	`text.count`
Index	`text[text.startIndex]`, `text[text.index(before: text.endIndex)]`
Prefix / suffix	`text.hasPrefix("ab")`, `text.hasSuffix("cd")`
Case	`text.uppercased()`, `text.lowercased()`
Splitting	`text.split(separator: ",")` returns substrings
Joining	`["a", "b"].joined(separator: "-")`
Reversing	`String(text.reversed())`
Characters	`Array("abc")` gives `["a", "b", "c"]`

`String` is a value type: assignment copies, and a `Character` is a grapheme cluster, so `"e" + an acute accent` counts as one character even though it is made of two Unicode scalars. String indices are not integers; use `String.Index` values obtained from the properties and methods of the string.

Tuples

A tuple is a lightweight product type: a fixed list of values, each with its own type, optionally with labels. Tuples are useful for returning several values from a function, and for pattern matching in `switch` cases. Named elements are read by name, unnamed elements by position, and both can be destructured with `let`.

`typealias` and `Any`

`typealias` creates an alias, not a new type; values of the alias and the original type are interchangeable. `Any` is an escape hatch for heterogeneous data; prefer concrete types or protocols, because a value of type `Any` cannot be used until it is cast back to something specific.

2.3 Examples

Example 1: constants, variables and annotations.

Swift

```swift
let birthYear = 1990
var temperature: Double = 21.5
var isRaining = false
let greeting = "Hello"
let label = "temperature"
print(birthYear, temperature, isRaining, greeting, label)
temperature = 22.0
isRaining = true
print(temperature, isRaining)
```

output

```
1990 21.5 false Hello temperature
22.0 true
```

Example 2: numeric literals and conversions.

Swift

```swift
let big = 1_000_000
let binary = 0b1010
let octal = 0o17
let hex = 0xFF
let exponent = 1.5e3
print(big, binary, octal, hex, exponent)
print(String(255, radix: 2), String(255, radix: 16))
print(Int("42") as Any)
print(Int("4x2") as Any)
print(Double("4.5") as Any)
```

output

```
1000000 10 15 255 1500.0
11111111 ff
Optional(42)
nil
Optional(4.5)
```

Literal separators are ignored, and the prefixes choose the base. Text to number conversions return an optional, because text is not always a valid number; `as Any` is used so the whole optional is printed instead of its contents.

Example 3: integer and floating point division, and truncation.

Swift

```swift
print(7 / 2)
print(7.0 / 2.0)
print(Int(3.9), Int(-3.9))
print(Double(3))
print(7 % 3, -7 % 3)
print(2.5.truncatingRemainder(dividingBy: 1.0))
```

output

```
3
3.5
3 -3
3.0
1 -1
0.5
```

Integer division truncates, so `7 / 2` is `3`. The remainder operator returns a result with the sign of the left operand, so `-7 % 3` is `-1`.

Example 4: booleans and comparisons.

Swift

```swift
let score = 87
let passed = score >= 60
print(passed, !passed, passed && score < 90, passed || score < 50)
let name = ""
print(name.isEmpty)
```

output

```
true false true true
true
```

`name.isEmpty` reads the property instead of comparing the string with `""`, which is cheaper and clearer.

Example 5: strings.

Swift

```swift
let language = "Swift"
let punctuation = "!"
var message = language + " is compiled"
message += punctuation
print(message)
print(message.count, message.uppercased())
print(language.hasPrefix("Sw"), language.hasSuffix("ft"))
print("a,b,c".split(separator: ","))
print(["a", "b", "c"].joined(separator: "-"))
print("Swift".hasPrefix("fast"))
```

output

```
Swift is compiled!
21 SWIFT IS COMPILED!
true true
["a", "b", "c"]
a-b-c
false
```

Example 6: characters and indices.

Swift

```swift
let letters = Array("swift")
print(letters)
let text = "swift"
print(text[text.startIndex])
print(text[text.index(before: text.endIndex)])
print(text.count)
let combined = "e\u{301}"
print(combined, combined.count, combined.unicodeScalars.count)
```

output

```
["s", "w", "i", "f", "t"]
s
t
5
é 1 2
```

The last line shows a grapheme cluster: one `Character` built from two scalars. The printed name of `String.Index` values is a hex offset, so index values are shown through the characters they select rather than printed directly.

Example 7: tuples.

Swift

```swift
let response = (status: 200, body: "ok")
print(response.status, response.0)
let pair = (1, "one")
print(pair.0, pair.1)

func range() -> (Int, Int) {
    return (minimum: 5, maximum: 9)
}
let result = range()
print(result.minimum, result.maximum)
let (minimum, maximum) = result
print(minimum, maximum)
```

output

```
200 ok
1 one
5 9
5 9
```

Both element names and positions work, and a tuple can be destructured into separate constants in one step.

Example 8: typealias, `Any` and `type(of:)`.

Swift

```swift
typealias Identifiers = [String]
let ids: Identifiers = ["ada", "grace"]
print(ids, type(of: ids))

var anything: Any = 42
anything = "forty two"
print(anything)
if let text = anything as? String {
    print("it was a string: \\(text)")
}
```

output

```
["ada", "grace"] Array<String>
forty two
it was a string: forty two
```

`Any` accepts both values, but each use needs a cast back to a concrete type.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assigning an `Int` to a `Double` variable	the compiler rejects it, so beginners add `!` or `as` and get further errors	write `Double(value)` explicitly
Using an integer index into a string	`String.Index` is not an integer; the code does not compile	use `startIndex`, `endIndex`, `index(after:)` and friends
Writing `if count` instead of `if count > 0`	Swift requires a `Bool`, so the code does not compile	compare explicitly

Mistake	Why it hurts	Best practice
Using `var` for every declaration	the reader must check whether the value changes	use `let` unless you assign again
Reaching for `Any`	type checking disappears at that point	use a concrete type, a protocol or a generic parameter
Mixing `Float` and `Double`	double conversions produce small rounding differences	stay with `Double` unless memory matters
Comparing floating point values with `==`	rounding makes exact equality fragile	compare with an explicit tolerance or compare rounded values

- Prefer `Int` and `Double` over the sized relatives unless you are writing code that stores or transmits exact binary layouts.
- When a type annotation feels necessary, add it: it documents intent, fixes inference, and shows up in the editor when you hover over the name.

2.5 Practice exercises

1. Convert `Int` 7, `Double` 7.0 and `String` "7" into each other, writing the conversions that are legal and naming those that are not.
2. Why does `var total: Double = 0` followed by `total = total + 2` fail, and what is the fix?
3. Write a tuple with a `String` label and an `Int`, then print both elements with and without using the label.
4. What is the value of `"a\u{301}".count` and of `"a\u{301}".unicodeScalars.count`?
5. What is the type of `let value = 0x1F` and of `let other = 1_000_000.0`?

Solutions

1. `Int(7.0)` gives `7`, `String(7)` gives `"7"`. `Double(7)` gives `7.0`. `"7"` must be converted with `Int("7")`, which returns an optional because the text may not be a number at all. There is no shorthand such as `7 as String`.
2. `total + 2` mixes a `Double` and an `Int`, so the compiler rejects it; write `total + 2.0` or `total + Double(2)`.
3. One solution:

Swift

```swift
let entry = (name: "Ada", score: 98)
print(entry.name, entry.score)
print(entry.0, entry.1)
```

output

```
Ada 98
Ada 98
```

4. `1` and `2`: the two scalars form one grapheme cluster, which is one `Character`.

5. `Int` and `Double`. A hexadecimal literal without a fractional part is an integer; a literal with a decimal point is a `Double`.

2.6 Mini test

1. Which declaration allows the value to change later? A) `let` B) `var` C) `const` D) `static`
2. What is the result of `Int(4.9)`? A) 5 B) 4 C) A runtime error D) An optional
3. Which of these is a valid condition for `if`? A) `if 3 { }` B) `if "yes" { }` C) `if isReady { }` where `isReady` is a `Bool` D) `if items { }` where `items` is an array
4. What does `[1, 2] + [3]` produce? A) `[1, 2, 3]` B) A runtime error C) `[1, 2]` D) `[[1, 2], [3]]`
5. What is `print(7 % 3, 7.0.truncatingRemainder(dividingBy: 3.0))`? A) `1 1` B) `1 1.0` C) `1 0.5` D) `2 1.0`
6. True or false: `String` is a value type in Swift.
7. Why is `"abc"[1]` not valid Swift? A) Strings cannot be subscripted B) Indices are `String.Index` values, not integers C) The subscript needs a range D) It is valid, and returns `b`
8. What does `print(7 / 2, 7.0 / 2.0)` print?
9. Which value prints as `Int` when passed to `type(of:)`? A) `7` B) `7.0` C) `"7"` D) `[7]`
10. What is the relationship between a `typealias` and its target? A) A new distinct type B) Another name for the same type C) A subtype D) A protocol requirement

Answer key and explanations

1. B - `let` creates a constant and `var` creates a variable.
2. B - conversion from a floating point value to an integer truncates toward zero.
3. C - only a `Bool` can be used as a condition, and only the third option is a `Bool`.
4. A - `+` on arrays concatenates them into a new array.
5. B - `%` on integers gives `1`, and the floating point remainder of `7.0` divided by `3.0` is `1.0`.
6. True - strings are copied on assignment and mutations are isolated, which is why sharing them is cheap and safe.
7. B - `String` indexes are opaque values obtained from the string; use `index(startIndex, offsetBy: 1)` to move along a string.
8. `3 3.5` - integer division truncates while floating point division keeps the fraction.
9. A - `7` is inferred as `Int`, while `7.0` is a `Double`.
10. B - an alias is interchangeable with its target; no new type is created.

Chapter 3 - Optionals

3.1 Summary

- An optional is a value that may be absent, written by adding `?` to a type.

- In Swift, `nil` is not a pointer and not a reference to an object; it means "no value at all".
- `Optional` is a real enumeration with the cases `some(Wrapped)` and `none`, and `?` is shorthand for it.
- A value of a non-optional type always contains a value, which is what lets the compiler reason about the rest of the program.
- Unwrap an optional with `if let`, `guard let`, a pattern in `switch`, `if case`, or by supplying a default with `??`.
- `??` evaluates its right-hand side only when the left-hand side is `nil`.
- Optional chaining with `?` returns an optional for the whole chain, even if the final member is not optional.
- Force unwrapping with `!` crashes the program when the optional is empty, so it is a statement of certainty, not a tool of convenience.
- An implicitly unwrapped optional, written with `!`, is still an optional, but it unwraps automatically at each use.
- Converting from an uncertain source such as text or a network response naturally produces optionals, and working with them is ordinary, everyday Swift.

3.2 Concepts

What an optional is

output

```
var middleName: String?    either a String or nil
var city: String           always a String, never nil
```

The optional form is the same type with a missing case added. Because the compiler tracks that distinction, the code that reads `city` never has to check for absence, and the code that reads `middleName` must.

`Optional` is an enumeration

output

```
enum Optional<Wrapped>: Equatable {
    case none            nil
    case some(Wrapped)   a value
}
```

That is why every optional supports the enumeration operations: you can `switch` over it, use `if case .some(let value) = optional`, and use the optional pattern `if case let value? = optional`.

Choosing an unwrapping strategy

Technique	Behaviour	Typical use
`if let x = opt`	runs the block when a value exists	work that only makes sense with a value

Technique	Behaviour	Typical use
`guard let x = opt else { ... }`	exits the scope when the value is missing	a function that cannot continue without it
`opt ?? default`	returns the value or the default	display text, empty collections, zero
`opt?.member`	returns an optional for the whole chain	long access paths through optional objects
`opt!`	crashes when `nil`	almost never; only for documented invariants
`switch opt`	exhaustive with `some` and `none`	when both cases need different handling

Several bindings may be joined in one `if let` or `guard let` statement with commas; the conditions are evaluated in order and short-circuit. Swift 5.7 added the shorthand that omits the right-hand side when a constant of the same name already exists.

Nil-coalescing

`a ?? b` unwraps `a` when it holds a value and otherwise evaluates `b`. Because the right-hand side is lazy, it can be an expensive expression or a function call that only runs when needed. Chains such as `a ?? b ?? c` read from left to right and use `c` only when both earlier values are missing.

Optional chaining

Chaining with `?` stops at the first missing link and makes the result of the whole expression optional. If the final member returns a non-optional value, the chain still yields an optional, because the link before it may have been missing. Assignment through a chain does nothing when a link is `nil`, which lets one statement update an optional object without opening an `if` block.

Implicitly unwrapped optionals

output

```
    var label: UILabel!      an optional that is unwrapped automatically
```

They exist for values that are `nil` while an object is being built and hold a value afterwards, such as outlets connected when a view loads. Writing the plain optional type is almost always better, because it keeps the check in the code instead of hiding it in the type.

Sources of optionals

Source	Result
`Int("42")`, `Double("1.5")`	`Optional` because the text may be invalid
A dictionary subscript	`Optional` because a key may be missing
A throwing call wrapped in `try?`	`Optional` because the call may throw
A failable initialiser `init?`	`Optional` because construction may fail

Source	Result
`first` and `last` on an empty collection	`Optional`
A view controller or outlet not yet loaded	`Optional`

3.3 Examples

Example 1: declaring and printing optionals.

Swift

```swift
var middleName: String?
print(middleName as Any)
middleName = "Grace"
print(middleName as Any)
print(middleName ?? "none")
let middleIsMissing = middleName == nil
print(middleIsMissing)
```

output

```
nil
Optional("Grace")
Grace
false
```

Printing an optional directly shows `Optional("Grace")`, so `as Any` makes the diagnostic output explicit. `nil` prints as `nil`.

Example 2: `if let`, several bindings and the shorthand.

Swift

```swift
let city: String? = "Buenos Aires"
let country: String? = "Peru"
if let city {
    print("city: \(city)")
}
if let city, let country {
    print("\(city), \(country)")
}
if let city = city, city.count > 4 {
    print("long name: \(city)")
}
```

output

```
city: Buenos Aires
Buenos Aires, Peru
long name: Buenos Aires
```

The second form binds both values only when both exist. The third form binds and then tests the bound value, and the new constant shadows the optional.

Example 3: `guard` and an early exit.

Swift

```swift
func describe(urlText: String?) -> String {
    guard let urlText else {
        return "no address"
    }
    guard urlText.hasPrefix("https://") else {
        return "unsafe address"
    }
    return "connecting to \\(urlText)"
}
print(describe(urlText: "https://swift.org"))
print(describe(urlText: "http://swift.org"))
print(describe(urlText: nil))
```

output

```
connecting to https://swift.org
unsafe address
no address
```

After the `guard` succeeds, the unwrapped value is available for the rest of the function, which avoids nesting the whole body inside an `if let`.

Example 4: defaults, chaining and comparisons.

Swift

```swift
let nickname: String? = nil
let handle = nickname ?? "anon"
print(handle)
let firstName: String? = "Ada"
let lastName: String? = "Lovelace"
print(firstName ?? "?" , lastName ?? "?")
print(firstName?.count ?? 0)
let empty: String? = ""
print(empty?.isEmpty ?? false)
```

output

```
anon
Optional("Ada") Optional("Lovelace")
3
true
```

Each line shows a value: `??` has replaced the missing nickname, the optionals were unwrapped while printing, and the count of "Ada" is 3.

Example 5: optional chaining through nested types.

Swift

```swift
struct Person {
    var name: String
    var address: Address?
}
struct Address {
    var city: String
}
let ada = Person(name: "Ada", address: Address(city: "London"))
let linus = Person(name: "Linus", address: nil)
print(ada.address?.city ?? "unknown")
print(linus.address?.city ?? "unknown")
print(ada.address?.city.count ?? -1)
```

output

```
London
unknown
6
```

The chained expression stops at the first missing link, and the default applies to the whole chain.

Example 6: pattern matching on optionals.

Swift

```swift
let values: [Int?] = [1, nil, 3, nil, 5]
for value in values {
    switch value {
    case .none:
        print("empty")
    case .some(let number):
        print("value: \\(number)")
    }
}
let candidate: Int? = nil
if case .none = candidate {
    print("nothing to do")
}
if case let number? = candidate {
    print("value: \\(number)")
} else {
    print("missing")
}
print(values.compactMap { $0 })
```

output

```
value: 1
empty
value: 3
empty
value: 5
nothing to do
missing
[1, 3, 5]
```

`compactMap` drops the `nil` entries and unwraps the rest, which is the usual way to go from an array of optionals to an array of values.

Example 7: optionals from conversions and throwing calls.

Swift

```swift
enum ParsingError: Error {
    case notANumber(String)
}
func parse(_ text: String) throws -> Int {
    guard let number = Int(text) else {
        throw ParsingError.notANumber(text)
    }
    return number
}
for text in ["12", "1x"] {
    do {
        print("parsed \\(try parse(text))")
    } catch ParsingError.notANumber(let bad) {
        print("cannot parse '\\(bad)'")
    }
}
let loose = try? parse("7")
print(loose as Any)
let absent = try? parse("x")
print(absent as Any)
```

output

```
parsed 12
cannot parse '1x'
Optional(7)
nil
```

`try?` turns a failure into `nil` instead of an error, which is convenient in scripts and in code that will try the next candidate.

Example 8: working with an optional inside a collection of structs.

Swift

```swift
struct Repository {
    var description: String?
    var stars: Int
}
let repos = [
    Repository(description: "Web framework", stars: 12_400),
    Repository(description: nil, stars: 90),
]
for repo in repos {
    let label = repo.description?.uppercased() ?? "UNTITLED"
    print("\\(label) - \\(repo.stars)")
}
```

output

```
WEB FRAMEWORK - 12400
UNTITLED - 90
```

`uppercased()` is called only when a description exists, and the default covers the missing case.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Force unwrapping with `!` to silence the compiler	a `nil` value crashes the program with no useful message	use `if let`, `guard let` or `??`
Using an implicitly unwrapped optional for convenience	the failure moves from the compile step to the runtime	declare the plain optional and unwrap it
Nested `if let` pyramids	the readable path disappears into indentation	join the bindings with commas, or use `guard`
Printing an optional and reading `Optional("x")` in the output	the diagnostic text is confusing	print with `as Any`, or unwrap before printing
Comparing an optional with a value directly	the types do not match and the code does not compile	unwrap first, or use `==` between optionals
Using `!` after a dictionary subscript because "the key exists"	the dictionary changes and the program crashes	use `?? default` or `if let`
Treating `nil` as a pointer or an object	Swift optionals are not Objective-C references	think of `nil` as "no value"

- `map` on an optional applies a function only when a value exists: `middleName?.count` is clearer, but `middleName.map { $0.count }` composes well with other functional code.
- When several optionals must all be present, a `guard` at the top of the function returns early and keeps the rest of the body flat.

3.5 Practice exercises

1. Write a function that takes an optional `Int` and returns its absolute value plus 10, or `-1` when the value is missing.
2. What is the printed output of `print(items.first?.count ?? 0)` when `items` is an empty array of `String`?
3. Explain the difference between `if let x = opt` and `guard let x = opt` in terms of the code that follows.
4. Given `let text: String? = nil`, write two ways to turn it into a `String` value that can be printed.
5. Why does the compiler reject `let n: Int = opt` when `opt` is an `Int?`, and which three changes make the code compile?

Solutions

1. One solution:

Swift

```swift
func value(_ number: Int?) -> Int {
    guard let number else { return -1 }
    return abs(number) + 10
}
print(value(3))
print(value(nil))
```

output

```
13
-1
```

2. `0` - `first` on an empty array is `nil`, so the `??` default is used and the optional chain never dereferences a missing element.
3. `if let` makes the unwrapped value available inside the block only, while `guard let` requires an exit of the current scope when the value is missing and makes the unwrapped value available after the `guard` statement.
4. Provide a default (`text ?? ""`), or unwrap with `if let` / `guard let`; `text!` also compiles but crashes if the optional is empty.
5. `Int?` and `Int` are different types, so an optional cannot be assigned to a non-optional binding. Use `let n: Int = opt ?? 0`, or `if let n = opt { ... }`, or change the type of `n` to `Int?`.

3.6 Mini test

1. Which type may hold `nil`? A) `String` B) `String?` C) `Int?` and `String?` D) only `Any`
2. What is the result of `nil ?? 5`? A) `nil` B) `5` C) A compile error D) `Optional(5)`
3. Which construct exits a function when a value is missing? A) `if let` B) `guard let ... else { }` C) `switch` D) `??`
4. What does `person.address?.city` return when `person.address` is `nil`? A) An empty `String` B) `nil` C) A crash D) `Optional("")`
5. True or false: `let x: String = Optional("hi")` compiles.
6. What does `try?` produce from a function that returns `String`? A) `String` B) `String?` C) `Result<String, Error>` D) It never fails
7. Which expression removes the `nil` entries from `[Int?]`? A) `values.map { $0 }` B) `values.compactMap { $0 }` C) `values.filter { $0 != nil }` D) `values.compactMap { $0 ?? 0 }`
8. What is the danger of the force unwrap operator?
9. What is the printed result of this code: `let v: Int? = 0; print(v as Any)`?
10. Which of the following is a correct way to test that an optional is empty? A) `if opt == nil { }` B) `if opt.isEmpty { }` C) `if opt == 0 { }` D) `if !opt { }`

Answer key and explanations

1. C - only the optional forms, written with `?`, can hold `nil`.
2. B - the left operand has no value, so the right operand is evaluated.
3. B - `guard` must leave the scope in its `else` branch.

4. B - optional chaining yields an optional for the whole chain, so a missing link gives `nil`.
5. False - an optional must be unwrapped before it can be assigned to a non-optional binding.
6. B - `try?` converts a thrown error into `nil` and keeps the success value, so the result type is `String?`.
7. B - `compactMap` discards the `nil` elements; `filter` keeps them as optionals.
8. It crashes the program when the optional is empty, and the failure happens at runtime with no compile-time warning that could have caught it.
9. `Optional(0)` - the whole optional is printed, which shows why unwrapping before `print` gives clearer diagnostics for zero values.
10. A - optionals compare with `nil`; the other forms assume the type inside the optional, and `!` is not a negation operator for optionals.

Chapter 4 - Operators and expressions

4.1 Summary

- Swift has the familiar arithmetic, comparison and logical operators, plus operators that protect against overflow and bit-level mistakes.
- Integer division truncates toward zero, and the remainder operator returns a result with the sign of the left-hand operand.
- Compound assignment operators such as `+=` combine an operation with a store and cannot be used with a `let` constant.
- Overflow operators `&+`, `&-` and `&*` trap nothing and wrap around instead, which is what bit-exact code needs.
- Comparison operators return a `Bool`, and `==` compares values while `===` compares identity between reference instances.
- Logical `&&` and `||` short-circuit, so the right-hand expression is evaluated only when it is needed.
- The ternary conditional operator `condition ? a : b` selects one of two expressions, and nil-coalescing `a ?? b` selects a fallback.
- Ranges are first-class values: closed `a...b`, half-open `a..<b`, and the one-sided forms `a...` and `...b`.
- Bitwise operators work on the binary representation of integers and are useful for flags and low-level protocols.
- Custom operators can be declared, but better code usually names a function instead.

4.2 Concepts

The operator groups

Group	Operators	Notes		
Arithmetic	`+ - * / %`	`/` truncates for integers		
Unary	`-x`, `!flag`	a space is not allowed between a unary `-` and its operand in most code styles		
Compound assignment	`+= -= *= /= %= &=	= ^= <<= >>=`	requires a mutable target	
Overflow	`&+ &- &*`	wrap around instead of trapping		
Comparison	`< <= > >= == !=`	any type that conforms to `Comparable` or `Equatable`		
Identity	`=== !==`	only for reference types		
Logical	`&&		!`	short-circuit, operands must be `Bool`
Ternary	`?:`	exactly one of two expressions		
Nil-coalescing	`??`	unwraps an optional or evaluates a default		
Range	`... ..<`	closed, half-open, one-sided		
Bitwise	`&	^ ~ << >>`	operate on the binary representation	
Pattern	`~=`	implemented by `Equatable` and by ranges		
Cast	`is`, `as`, `as?`, `as!`	type checks and casts, see chapter 7		

Integer arithmetic

output

```
7 / 2      = 3      integer division truncates
7.0 / 2.0  = 3.5    floating point division keeps the fraction
7 % 3      = 1      remainder with the sign of the left operand
-7 % 3     = -1
```

Overflow on a fixed-width integer traps at run time: `Int8.max + 1` crashes the program. The wrapping operators exist for the cases where wraparound is the intended behaviour, and the standard library also provides `addingReportingOverflow()` when you want to inspect the result without crashing.

Ranges

output

```
1...5       1, 2, 3, 4, 5        closed
1..<5       1, 2, 3, 4           half-open
20...       from 20 to the end   one-sided, countable
..<3        everything before 3  one-sided
5...        empty when the collection has fewer than 5 elements
```

A range is a value: it can be stored in a `let`, passed to a function, and used as a collection subscript. One-sided ranges work on any `Collection`, and countable ranges can be iterated with `for`.

Bitwise operators

Expression	Binary	Value	
`0b1010 & 0b0110`	`0010`	`2`	
`` `0b1010 ``	`0b0110` ``	`1110`	`14`
`0b1010 ^ 0b0110`	`1100`	`12`	
`~0b1010`	`...0101`	`-11`	
`0b1010 << 2`	`101000`	`40`	
`0b1100 >> 2`	`0011`	`3`	

Bitwise operations are the natural way to build and test flag sets, for example when checking a set of permissions packed into one integer.

Short-circuiting

output

```
if user != nil && user!.isLoggedIn { }   the second part runs only if the first is true
```

This matters for correctness as well as speed: the right-hand side can rely on the left-hand side being true, which is the idea behind the `if let x = a, x > 0` combination seen in chapter 3.

Precedence and custom operators

Swift has no operator precedent from the language grammar: precedence is attached to operators through precedence groups, which declare how an operator associates (`left`, `right`, `none`) and its precedence relative to other groups. The standard groups, from tightest to loosest, are `BitwiseShiftPrecedence`, `MultiplicationPrecedence`, `AdditionPrecedence`, `RangeFormationPrecedence`, `CastingPrecedence`, `NilCoalescingPrecedence`, `ComparisonPrecedence`, `LogicalConjunctionPrecedence` and `LogicalDisjunctionPrecedence`.

A custom operator is declared like this:

Swift

```swift
infix operator !!: AdditionPrecedence
func !! (left: Int, right: Int) -> Int { left * left + right * right }
```

Naming a function is usually clearer than inventing an operator, so custom operators are best reserved for domain-specific notation that a reader of the code already knows.

4.3 Examples

Example 1: arithmetic, assignment and overflow.

Swift

```swift
var total = 10
total += 5
total -= 3
total *= 2
print(total)
print(9 / 2, 9 % 2)
print(9.0 / 2.0, 9.0.truncatingRemainder(dividingBy: 2.0))
print(Int8.max)
print(Int8.max &+ 1)
```

output

```
24
4 1
4.5 1.0
127
-128
```

The last line is the wrapping addition: one past the maximum of `Int8` is its minimum. The plain `+` version of the same expression traps at run time.

Example 2: comparison, identity and logic.

Swift

```swift
struct Point: Equatable {
    var x: Int
    var y: Int
}
let a = Point(x: 1, y: 2)
let b = Point(x: 1, y: 2)
print(a == b)

class Box {
    var value: Int
    init(value: Int) { self.value = value }
}
let box1 = Box(value: 1)
let box2 = box1
let box3 = Box(value: 1)
print(box1 === box2, box1 === box3)

let age = 20
print(age >= 18 && age < 30, age < 18 || age >= 30, !(age >= 18))
```

output

```
true
true false
true false false
```

Two values of an `Equatable` struct can be equal, while two instances of a class are identical only when they are the same object.

Example 3: the ternary operator and nil-coalescing.

Swift

```swift
let temperature = 31
let weather = temperature > 30 ? "hot" : "mild"
print(weather)
let requested: Int? = nil
let perPage = requested ?? 20
print(perPage)
let city: String? = nil
let country: String? = "Peru"
print(country ?? city ?? "nowhere")
```

output

```
hot
20
Peru
```

The nil-coalescing chain only uses the later values when the earlier ones are missing.

Example 4: ranges and `stride`.

Swift

```swift
let numbers = Array(1...5)
print(numbers)
print(Array(1..<5))
print(Array(stride(from: 0, through: 10, by: 5)))
print(Array(stride(from: 0, to: 10, by: 3)))
for index in 0..<numbers.count {
    print(index, terminator: " ")
}
print()
let letters = ["a", "b", "c", "d"]
print(letters[1...2], letters[...2], letters[1...])
```

output

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4]
[0, 5, 10]
[0, 3, 6, 9]
0 1 2 3 4
["b", "c"] ["a", "b", "c"] ["b", "c", "d"]
```

`stride(from:through:)` includes the end value while `stride(from:to:)` does not. One-sided ranges as subscripts select slices from the start or the end.

Example 5: bitwise operators and flags.

Swift

```swift
let read = 0b100    // a flag holding 4
let write = 0b010   // a flag holding 2
let exec = 0b001    // a flag holding 1
var permissions = read | write
print(permissions)
permissions |= exec
print(permissions)
let canExecute = permissions & exec == exec
print(canExecute)
print(permissions & ~exec)
print(1 << 4, 32 >> 2)
print(0b1010 ^ 0b1010)
```

output

```
6
7
true
6
16 8
0
```

`& ~exec` clears one bit, and the exclusive-or combination cancels itself out, which is a common trick for toggling a flag.

Example 6: pattern matching with `~=` and ranges in `switch`.

Swift

```swift
for score in [12, 45, 87, 100] {
    switch score {
    case 0..<50:
        print("low")
    case 50..<100:
        print("high")
    default:
        print("perfect")
    }
}
print(5 ~= 1...10)
print("hello" ~= "hello")
```

output

```
low
low
high
perfect
true
true
```

Ranges and equality both provide the `~=` operator, which is what lets a `switch` match a value against a range or a literal.

Example 7: `defer` of operator work - expression order.

Swift

```swift
var results: [Int] = []
let x = 2
let y = 3
results.append(x * y + 1)
results.append(x + y * 2)
print(results)
let z = results[0] > results[1] ? results[0] : results[1]
print(z)
print(2 + 3 * 4 - 6 / 2)
print(2 * 3 + 4 * 5)
```

output

```
[7, 8]
8
11
26
```

Multiplication binds tighter than addition, and division is evaluated before subtraction. Parentheses make intent explicit and are never wrong.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice		
Expecting `7 / 2` to be `3.5`	integer division truncates, so the result silently loses precision	divide `Double` values when a fraction matters		
Using `+` where `&+` is meant in bit-level code	a trap crashes the program	use the wrapping operators and document why		
Comparing two class instances with `==`	the meaning is unclear and the class may not be `Equatable`	use `===` for identity, `Equatable` for value equality		
Writing `if a != nil && a.count > 0`	force unwraps and repeats the check	use `if let a, a.count > 0`		
Assuming `		` evaluates both sides	a side effect on the right may never run	keep the short-circuit behaviour in mind, avoid hidden side effects
Building long operator chains without parentheses	readers cannot tell the evaluation order	add parentheses where a human would pause		

Mistake	Why it hurts	Best practice
Inventing a custom operator for a library	readers must learn a new symbol	prefer a well-named function
Using `<<` and `>>` on signed values for arithmetic	sign extension surprises	use `*` and `/`, or unsigned types for bit work

- Prefer `stride` or a range over a hand-written `while` loop when stepping through values: the loop variable is immutable and the bounds are explicit.
- When an operator's behaviour is not obvious to a reader of your code, wrap it in a function whose name says what it computes.

4.5 Practice exercises

1. What is the value of `10 % 4`, `-10 % 4`, `11 / 2` and `11.0 / 2.0`?
2. Write an expression that is `true` when the integer `flags` has bit 2 set.
3. Rewrite `if count != 0 && name != nil` using optionals correctly.
4. What is the output of `print(Array(0..<4).reversed())` and of `print(Array(stride(from: 4, through: 4, by: 1)))`?
5. Explain the difference between `===` and `==`, and name the type each one requires.

Solutions

1. `2`, `-2`, `5` and `5.5`: the remainder takes the sign of the left operand, integer division truncates, and floating point division keeps the fraction.
2. `flags & 0b100 == 0b100` or, in hexadecimal, `flags & 0x4 == 0x4`.
3. `if count != 0, let name { ... }` - a binding, not a comparison; a comparison against `nil` plus a force unwrap is the mistake.
4. `[3, 2, 1, 0]` and `[4]` - `reversed()` returns a reversed view that is copied into an array by the initialiser, and a stride whose start equals its end through value contains exactly that one value.
5. `===` compares identity and requires reference types (classes, closures and object references); `==` compares values and requires conformance to `Equatable`.

4.6 Mini test

1. What is `9 / 2` in Swift? A) `4.5` B) `4` C) `5` D) A compile error
2. What does the wrapping addition do when `UInt8` overflows? A) Traps and stops the program B) Continues from the minimum value C) Returns an optional D) Rounds to the nearest power of two
3. Which operator tests whether two class instances are the same object? A) `==` B) `===` C) `~=` D) `!==`

4. What is `1...4` as a range? A) `1, 2, 3` B) `1, 2, 3, 4` C) `1, 2, 3, 4, 5` D) An empty range
5. Which expression is `true` when `permissions` contains the bit of value 4? A) `permissions & 4 != 0` B) `permissions | 4 != 0` C) `permissions ^ 4 == 4` D) `permissions << 4 == 4`
6. True or false: `&&` evaluates its right-hand operand only when the left-hand operand is `true`.
7. What does `print(2 + 3 * 2)` print? A) `10` B) `8` C) `12` D) `7`
8. Which operator supplies a default for an optional? A) `??` B) `?.` C) `??=` D) `!=`
9. What is the value of `~0`? A) `0` B) `-1` C) `1` D) The maximum integer
10. Which compound assignment is legal on a `let` constant?

Answer key and explanations

1. B - `4`; both operands are integers, so the division truncates.
2. B - the wrapping arithmetic continues from the minimum, which is why `UInt8.max &+ 1` is `0`.
3. B - `===` compares identity; `!==` is its negation.
4. B - a closed range includes both endpoints, so it holds four values.
5. A - the bitwise-and with the flag is non-zero exactly when the flag is present.
6. True - the operator short-circuits, so the right-hand side is skipped when the result is already known.
7. B - multiplication binds tighter than addition, so the expression is `2 + 6`.
8. A - `??` unwraps the optional or evaluates its right-hand side.
9. B - complementing zero flips every bit, which is the all-ones pattern representing `-1` in two's complement.
10. None of them: compound assignment stores a new value, so it requires a `var` binding.

Chapter 5 - Control flow and early exits

5.1 Summary

- Swift has the familiar `if`, `else if`, `else`, `while` and `repeat-while` statements, and every condition must be a `Bool`.
- `switch` has no implicit fallthrough, must be exhaustive, and can match values, ranges, tuples, bindings and `where` conditions.
- `for in` iterates over arrays, dictionaries, ranges, strides, sequences and pairs produced by `enumerated()` or `zip`.
- `where` clauses filter loop iterations without nesting an `if` inside the loop.
- `break` leaves a loop or a `switch` case, `continue` skips to the next iteration, and labels target an outer loop.
- `guard` is a statement, not a conditional: it either lets execution continue with a bound value or it must leave the current scope.

- `defer` registers a block that runs when the scope exits, in reverse order, including after a thrown error.
- Pattern matching forms such as `if case`, `guard case` and `for case` give `switch` power without the full statement.
- Because `switch` over an enumeration must cover every case, adding a case to the enumeration makes the compiler list every place that must be updated.
- Swift has no unconditional `goto`; loops are structured, which keeps the flow of a function readable.

5.2 Concepts

The conditional statements

Statement	Meaning
`if condition { }`	runs when the condition is true
`if a { } else if b { } else { }`	the first true branch runs
`while condition { }`	repeats while the condition holds, zero or more times
`repeat { } while condition`	runs the body once, then repeats while the condition holds
`guard condition else { }`	leaves the scope unless the condition holds

`guard` is followed by code that can only run when the condition is true, and the `else` branch must transfer control out of the enclosing scope with `return`, `break`, `continue` or `throw`. It also binds values the same way `if let` does, which is what makes it useful.

`switch` rules

output

```
switch value {
case pattern1:
    code
case let x where x > 10:
    code
default:
    code
}
```

1. Patterns are tested in order, and the first match wins.
2. There is no fallthrough, so each case ends by itself.
3. The `default` case covers everything that the other cases do not.
4. A case matches a value, a range, a tuple pattern, a binding or a `where` condition.
5. When the switch covers an enumeration, the compiler knows when all cases are covered and the `default` becomes optional.
6. A case does not fall through even when its body is empty; an empty body simply does nothing.

The pattern forms

Pattern	Example
Expression	`case 0`, `case "hello"`
Range	`case 1...10`, `case ..<0`
Tuple	`case (0, 0)`
Value binding	`case let (x, y)`
Where condition	`case let n where n > 0`
Enumeration case	`case .downloading(let progress)`
Optional pattern	`case let value?`
Boolean pattern	`case true`
Type cast	`case let s as String`
`is` pattern	`case is String`

Loops

Form	Typical use
`for item in collection`	visits every element
`for key in dictionary`	visits the keys
`for (key, value) in dictionary`	visits key-value pairs
`for (index, value) in array.enumerated()`	visits elements with their position
`for i in stride(from:to:by:)`	steps by a value other than one
`for (a, b) in zip(first, second)`	walks two sequences in parallel
`for value in numbers where value > 0`	visits only the elements that pass
`while` / `repeat-while`	repeats until a condition changes

`zip` stops at the shorter sequence, and `enumerated()` returns a sequence of tuples whose first element is the offset from the start of the sequence.

`defer`

`defer` runs the block when control leaves the enclosing scope for any reason: falling off the end, a `return`, a `break`, a `throw`. Deferred blocks run in reverse order of registration, which makes them well suited to releasing a resource that was acquired earlier.

5.3 Examples

Example 1: conditional statements.

Swift

```swift
let temperature = 14
if temperature < 0 {
    print("freezing")
} else if temperature < 15 {
    print("cold")
} else if temperature < 25 {
    print("pleasant")
} else {
    print("hot")
}
let hasIce = true
let willFreeze = temperature < 0 && hasIce
print(willFreeze)

var countdown = 3
while countdown > 0 {
    print(countdown, terminator: " ")
    countdown -= 1
}
print()
repeat {
    print("runs once: countdown is \\(countdown)")
} while countdown > 0
```

output

```
cold
false
3 2 1
runs once: countdown is 0
```

Example 2: `switch` with ranges, `where` and tuples.

Swift

```swift
let point = (x: 0, y: 5)
switch point {
case (0, 0):
    print("origin")
case (let x, 0):
    print("on the x axis at \\(x)")
case (0, let y):
    print("on the y axis at \\(y)")
case (let x, let y) where x == y:
    print("diagonal at \\(x)")
default:
    print("point \\(point.x), \\(point.y)")
}

for temperature in [-5, 10, 18, 40] {
    switch temperature {
    case ...0:
        print("below freezing")
    case 1...15:
        print("cold")
    case 16...25:
        print("mild")
    default:
        print("warm")
    }
}
```

output

```
on the y axis at 5
below freezing
cold
mild
warm
```

The first `switch` matches `(0, 5)` against the third case, so the binding `y` holds 5.

Example 3: iterating collections, with `where` and `zip`.

Swift

```swift
let scores = ["Ada": 90, "Grace": 95, "Linus": 85]
for name in scores.keys.sorted() {
    print("\\(name): \\(scores[name] ?? 0)")
}

let words = ["map", "filter", "reduce"]
for (index, word) in words.enumerated() {
    print(index, word)
}

for (letter, number) in zip(["a", "b"], [1, 2, 3]) {
    print("\\(letter) -> \\(number)")
}

let values = [1, 2, 3, 4, 5, 6]
for value in values where value % 2 == 0 {
    print(value, terminator: " ")
}
print()
```

output

```
Ada: 90
Grace: 95
Linus: 85
0 map
1 filter
2 reduce
a -> 1
b -> 2
2 4 6
```

Dictionary iteration order is not defined, so the keys are sorted before printing to make the output stable.

Example 4: `break`, `continue` and labels.

Swift

```swift
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
var squared: [Int] = []
for number in numbers {
    if number == 5 {
        break
    }
    if number % 2 == 0 {
        continue
    }
    squared.append(number * number)
}
print(squared)

let rows = [[1, 0, 3], [4, 0, 6], [7, 8, 9]]
search: for row in rows {
    for cell in row {
        if cell == 0 {
            print("found a hole")
            break search
        }
    }
}
```

output

```
[1, 9]
found a hole
```

Without the label the inner loop would break, the outer loop would continue, and the search would keep looking.

Example 5: `guard` for an early exit.

Swift

```swift
func divide(_ dividend: Int, by divisor: Int) -> Int? {
    guard divisor != 0 else {
        print("no division by zero")
        return nil
    }
    guard divisor > 0 else {
        print("negative divisor")
        return nil
    }
    return dividend / divisor
}
print(divide(10, by: 2) as Any)
print(divide(10, by: 0) as Any)
print(divide(10, by: -2) as Any)

func firstPositive(in values: [Int]) -> Int? {
    for value in values {
        guard value > 0 else {
            continue
        }
        return value
    }
    return nil
}
print(firstPositive(in: [-3, -1, 4, 7]) as Any)
```

output

```
Optional(5)
no division by zero
nil
negative divisor
nil
Optional(4)
```

A `guard` in a loop can use `continue`, so the body below it only handles the case that passed.

Example 6: `defer`.

Swift

```swift
func process(order: String) {
    defer {
        print("close the order")
    }
    print("open \\(order)")
    guard !order.isEmpty else {
        print("nothing to do")
        return
    }
    defer {
        print("log the order")
    }
    print("work on \\(order)")
}
process(order: "")
process(order: "42")
```

output

```
open
nothing to do
close the order
open 42
work on 42
log the order
close the order
```

The deferred blocks run in reverse order of registration, and the first one registered still runs when the function returns early.

Example 7: pattern matching forms.

Swift

```swift
let optionalValues: [Int?] = [3, nil, 5]
for value in optionalValues {
    if case let number? = value {
        print("has \\(number)")
    }
    if case .some(let number) = value {
        print("again \\(number)")
    }
}
let reported: Int? = nil
print(reported ?? "not set")

enum Download {
    case idle
    case running(percent: Int)
    case failed(reason: String)
    case done(bytes: Int)
}
let state = Download.running(percent: 40)
switch state {
case .idle:
    print("waiting")
case .running(let percent):
    print("running \\(percent)%")
case .failed(let reason):
    print("failed: \\(reason)")
case .done(let bytes):
    print("done: \\(bytes) bytes")
}
```

output

```
has 3
again 3
has 5
again 5
not set
running 40%
```

`if case let number?` is the optional pattern form of the same test as `if case .some(let number)`.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using a number as a condition	the compiler rejects it, and adding `!` or `== nil` hides the intent	compare or unwrap explicitly
Forgetting the `default` case on a non-enumeration switch	the compiler rejects the switch as non-exhaustive	add `default` or cover every case
Expecting a `switch` case to fall through	each case ends by itself, so code after it never runs	write one case per behaviour, or combine patterns with a comma
Nesting an `if` inside a loop for filtering	indentation grows and the readable path shrinks	use a `where` clause or `continue`

Mistake	Why it hurts	Best practice
Writing a `guard` without an exit statement	the compiler rejects it	exit with `return`, `break`, `continue` or `throw`
Using `defer` for ordinary work that must happen before a return	the deferred work runs later than the reader expects	reserve `defer` for cleanup and release
Relying on dictionary iteration order	the order changes between runs and between platforms	sort the keys when order matters
Using labelled breaks everywhere	the flow becomes harder to follow than the loop it escapes	extract a function instead

- Prefer `switch` to a chain of three or more `if else if` tests: the compiler checks that the cases cover the domain.
- When a function grows a second level of nesting, look for a `guard` or an extracted function before adding another `if`.

5.5 Practice exercises

1. Write a `switch` over an `Int` value that prints "single" for 1, "small" for 2 and 3, "medium" for 4...6 and "large" otherwise.
2. What is the output of the following loop, and why?

Swift

```swift
for value in 1...5 where value % 2 == 0 {
    print(value)
}
```

3. Rewrite the following code using `guard`, without changing its behaviour.

Swift

```swift
func describe(name: String?) -> String {
    if let name {
        if name.count > 3 {
            return "long"
        } else {
            return "short"
        }
    } else {
        return "missing"
    }
}
```

4. Where does a `defer` block run when the function throws an error?
5. Which `for` loop form prints the index and the value of every element of `["x", "y"]`?

Solutions

1. One solution:

Swift

```swift
let size = 5
switch size {
case 1:
    print("single")
case 2, 3:
    print("small")
case 4...6:
    print("medium")
default:
    print("large")
}
```

output

```
medium
```

2. It prints `2` and `4` on separate lines. The `where` clause filters the iterations, so only the even values of the range are visited.
3. One solution:

Swift

```swift
func describe(name: String?) -> String {
    guard let name else { return "missing" }
    guard name.count > 3 else { return "short" }
    return "long"
}
```

4. Before the error leaves the function, after the rest of the scope has run; the `catch` site then handles it.
5. `for (index, value) in ["x", "y"].enumerated() { ... }`, which pairs each element with its offset from the start.

5.6 Mini test

1. Which condition is valid for `if` in Swift? A) `if score { }` where `score` is an `Int` B) `if isReady { }` where `isReady` is a `Bool` C) `if "text" { }` D) `if items { }` where `items` is an array
2. What happens when a `switch` case finishes its body? A) Execution continues into the next case B) The switch ends C) The program stops D) The default case runs
3. Which statement must a `guard`'s `else` branch contain? A) `break` or `continue` only B) A transfer of control out of the current scope C) `print` D) Nothing
4. What is the output of `for value in [1, 2, 3] where value > 1 { print(value) }`? A) `1 2 3` B) `2 3` C) `1` D) Nothing
5. When do `defer` blocks run? A) In registration order, at the start of the scope B) In reverse registration order, when the scope exits C) Only when an error is thrown D) After the program ends
6. True or false: a `switch` over an enumeration with three cases and no `default` compiles only if all three cases are listed.

7. What does `zip(["a", "b"], [1, 2, 3])` produce? A) Three pairs B) Two pairs C) A compile error D) An endless sequence
8. Which construct makes an unwrapped value available after the statement, instead of only inside a block? A) `if let` B) `switch` C) `guard let` D) `while let`
9. What is the output of `repeat { print("once") } while false`?
10. Which of the following is a tuple pattern in a `switch` case?

Answer key and explanations

1. B - a condition must be a `Bool`, and only the second option is one.
2. B - Swift's `switch` does not fall through, so each case ends by itself.
3. B - the `else` branch must leave the scope, otherwise the code after the `guard` would be unreachable.
4. B - the `where` clause keeps only the values that pass the condition.
5. B - deferred blocks run last-in, first-out, on every exit path.
6. True - a `switch` over an enumeration is exhaustive when every case is covered, so the `default` is not needed.
7. B - `zip` stops at the shorter sequence, so the third element has no partner.
8. C - `guard` binds the value for the rest of the scope that follows.
9. `once` - a repeat-while loop runs its body before testing the condition.
10. `case let (x, y):` - it binds both elements of a tuple; `case 0` is an expression, and `case .some(let v)` is an enumeration case pattern.

Chapter 6 - Functions and closures

6.1 Summary

- A function is declared with `func`, a name, a parameter list and an optional return type.
- Swift separates the argument label used at the call site from the parameter name used in the body, which reads like a sentence at the call site.
- Writing _ before a parameter name omits the argument label.
- Default values let a caller skip arguments, and variadic parameters collect zero or more values of one type into an array.
- `inout` parameters let a function change a value that belongs to the caller, and are passed with a leading `&`.
- A function may return a tuple when it has several results of one meaning.
- Functions are values: they can be stored in a constant, passed as an argument and returned from another function.
- A closure is a function without a name, written in braces, that can capture and keep values from the scope where it was created.

- Closures infer types from context, name their parameters `$0`, `$1` and so on, and can be written after the parentheses as a trailing closure.
- Higher-order functions such as `map`, `filter` and `reduce` take closures, which turns a loop into an expression that returns a new value.
- A closure that outlives the call that received it must be marked `@escaping`, which is how completion handlers work.

6.2 Concepts

Argument labels and parameter names

output

```
func greet(person name: String, greeting word: String) -> String
               ^       ^            ^          ^
               label   name          label      name

call site:  greet(person: "Ada", greeting: "Hello")
body:       uses name and word
```

The default in Swift is that every parameter has a label. Omitting it is a choice you make by writing _, and the convention is to omit it for values that carry no meaning at the call site, such as a first parameter in `insert(_ value: Int, at index: Int)`.

A summary of parameter forms

Form	Declaration	Call
With a label	`func move(to target: String)`	`move(to: "home")`
Without a label	`func move(_ target: String)`	`move("home")`
Custom name	`func set(_ value: Int, forKey key: String)`	`set(3, forKey: "x")`
Default value	`func log(_ text: String, level: Int = 0)`	`log("x")` or `log("x", level: 2)`
Variadic	`func sum(_ values: Int...)`	`sum(1, 2, 3)`
`inout`	`func bump(_ value: inout Int)`	`bump(&counter)`
No parameters	`func now() -> Int`	`now()`

Return values

A function without an explicit result returns `Void`. A function whose body is a single expression may omit `return`, and so may a closure with a single expression. When the result can be ignored, mark the function `@discardableResult` so callers do not get a warning.

Function types

output

```
(Int, Int) -> Int          takes two integers, returns one
() -> Void                 takes nothing, returns nothing
(String) -> [Int]          takes text, returns an array
(Int, Int) -> Int          can be stored in a constant and called later
```

A function type can appear anywhere a type appears, which makes it possible to pass behaviour as an argument and to build functions that build functions.

Closures

output

```
{ (parameters) -> ReturnType in
    statements
}

shorthand forms, from most explicit to shortest:
sorted(by: { (a, b) in a < b })
sorted(by: { a, b in a < b })
sorted { a, b in a < b }
sorted(by: <)
sorted { $0 < $1 }
```

A trailing closure is written after the parentheses of the call and is the Swift style choice when the closure is the last and largest argument. When several closures take arguments, the labels come first and the closures follow in order.

Capture and escape

A closure captures the values it uses:

Captured value	What the closure keeps
A value type	a copy of the value at creation
A reference type	the same object, not a copy
A `var` local	a shared box, so the closure and the scope see the same value

When a closure is stored, returned or passed to a task that runs later, the function call that received it finishes before the closure runs. That is what "escaping" means, and the compiler requires `@escaping` on such a parameter.

Higher-order functions

Function	Effect
`map`	builds a new collection with the transformed elements
`filter`	keeps the elements that pass a test
`reduce`	combines the elements into one value
`compactMap`	maps and drops the `nil` results
`flatMap`	maps each element to a sequence and concatenates
`sorted(by:)`	returns a new sorted array, leaving the original unchanged

Function	Effect
`contains(where:)`	answers whether any element passes
`first(where:)`	finds the first element that passes
`removeAll(where:)`	removes the elements that pass, in place
`forEach`	runs a closure for its side effects

6.3 Examples

Example 1: labels, defaults and a discarded result.

Swift

```swift
func greet(name: String, greeting: String = "Hello", punctuation: String = "!") -> String {
    return "\\(greeting), \\(name)\\(punctuation)"
}
print(greet(name: "Ada"))
print(greet(name: "Linus", greeting: "Welcome"))
print(greet(name: "Grace", greeting: "Hi", punctuation: "."))

@discardableResult
func record(_ value: Int, label: String) -> Int {
    print("\\(label): \\(value)")
    return value
}
record(7, label: "attempt")
let stored = record(8, label: "final")
print(stored)
```

output

```
Hello, Ada!
Welcome, Linus!
Hi, Grace.
attempt: 7
final: 8
8
```

Example 2: variadic parameters, `inout` and several results.

Swift

```swift
func sum(_ values: Int...) -> Int {
    return values.reduce(0, +)
}
print(sum())
print(sum(1, 2, 3, 4))

func swapValues<T>(_ a: inout T, _ b: inout T) {
    let temporary = a
    a = b
    b = temporary
}
var first = 1
var second = 2
swapValues(&first, &second)
print(first, second)

func statistics(_ values: [Int]) -> (minimum: Int, maximum: Int, average: Double) {
    let total = values.reduce(0, +)
    return (values.min()!, values.max()!, Double(total) / Double(values.count))
}
let stats = statistics([4, 8, 15, 16, 23, 42])
print(stats.minimum, stats.maximum, stats.average)
```

output

```
0
10
2 1
4 42 18.0
```

A variadic parameter with no arguments produces an empty array, so `sum()` returns the initial value of the reduction. The `inout` values are passed with `&` and are restored in the caller when the function returns.

Example 3: functions as values.

Swift

```swift
func add(_ a: Int, _ b: Int) -> Int { a + b }
func multiply(_ a: Int, _ b: Int) -> Int { a * b }

let operation: (Int, Int) -> Int = add
print(operation(3, 4))

func apply(_ operation: (Int, Int) -> Int, to a: Int, and b: Int) -> Int {
    return operation(a, b)
}
print(apply(add, to: 3, and: 4))
print(apply(multiply, to: 3, and: 4))

func makeOperation(kind: String) -> (Int, Int) -> Int {
    return kind == "add" ? add : multiply
}
print(makeOperation(kind: "add")(2, 5))
print(makeOperation(kind: "multiply")(2, 5))
```

output

```
7
7
12
7
10
```

The function type is the parameter of `apply`, so the same code performs two different operations.

Example 4: closures that transform collections.

Swift

```swift
let numbers = [1, 2, 3, 4]
print(numbers.map { $0 * $0 })
print(numbers.filter { $0 % 2 == 0 })
print(numbers.reduce(0, +))
print(numbers.reduce(1, *))
print(numbers.sorted { $0 > $1 })
print(numbers.sorted(by: >))
print([[1, 2], [3, 4]].flatMap { $0 })
print(numbers.map { (value: $0, square: $0 * $0) })
print(numbers.contains { $0 == 3 })
print(numbers.first { $0 > 2 } as Any)
```

output

```
[1, 4, 9, 16]
[2, 4]
10
24
[4, 3, 2, 1]
[4, 3, 2, 1]
[1, 2, 3, 4]
[(value: 1, square: 1), (value: 2, square: 4), (value: 3, square: 9), (value: 4, square: 16)]
true
Optional(3)
```

`sorted { $0 > $1 }` and `sorted(by: >)` are the same call written twice; the operator itself is a function value.

Example 5: a trailing closure with several arguments.

Swift

```swift
func load(id: Int, completion: (String) -> Void) {
    completion("row \\(id)")
}
load(id: 7) { text in
    print(text)
}

let labels = ["one", "two", "three"]
labels.forEach { label in
    print("> \\(label)")
}
labels.forEach {
    print("<\\($0)")
}
```

output

```
row 7
> one
> two
> three
<one
<two
<three
```

The second form uses the shorthand argument name `$0`, so no parameter list or `in` keyword is needed.

Example 6: capture and escape.

Swift

```swift
func makeCounter() -> () -> Int {
    var total = 0
    return {
        total += 1
        return total
    }
}
let counter = makeCounter()
print(counter(), counter(), counter())

func buildQueue(count: Int) -> [() -> Int] {
    var queue: [() -> Int] = []
    for index in 1...count {
        queue.append { index }
    }
    return queue
}
let tasks = buildQueue(count: 3)
for task in tasks {
    print(task(), terminator: " ")
}
print()

var queued: [() -> Void] = []
func addTask(_ body: @escaping () -> Void) {
    queued.append(body)
}
addTask { print("first") }
addTask { print("second") }
queued.forEach { $0() }
```

output

```
1 2 3
1 2 3
first
second
```

Each closure created in the loop keeps its own copy of the loop variable, so the queue produces 1, 2 and 3 rather than the same value three times.

Example 7: autoclosure and a function that takes a function.

Swift

```swift
func debugLog(_ message: @autoclosure () -> String) {
    print("LOG", message())
}
let name = "Ada"
debugLog("hello \\(name) \\(name.count)")

func repeatAction(times: Int, action: () -> Void) {
    for _ in 0..<times {
        action()
    }
}
repeatAction(times: 2) {
    print("hi")
}
func transform(_ values: [Int], using body: (Int) -> Int) -> [Int] {
    return values.map(body)
}
print(transform([1, 2, 3]) { $0 + 10 })
```

output

```
LOG hello Ada 3
hi
hi
[11, 12, 13]
```

The autoclosure turns the expression into a closure, so it is evaluated only when the function uses it.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Omitting the argument label at the call site out of habit	the call no longer reads like a sentence	name the label after the role of the value
Passing a `let` constant to an `inout` parameter	the compiler rejects it, and force-casting does not help	pass a `var` with `&`
Giving a parameter both a default value and a label when the label adds nothing	calls become noisy	use _ for parameters whose meaning is obvious
Writing a closure with an explicit type for every parameter	more syntax to read and maintain	let the context infer it, add types only for clarity
Writing `sorted` to change an array in place	the original is unchanged and the result is dropped	assign the result, or use `sort()` on the array
Storing a closure that is expected to be escaping without marking it	the compiler rejects the code	mark the parameter `@escaping` and document the ownership
Building a closure that captures `self` in a class hierarchy	a reference cycle keeps both objects alive	use a capture list, for example `[weak self]`
Using `map` when `forEach` is meant	building and discarding a new array wastes work	use `forEach` for side effects, `map` for transformation

- Name a closure's parameters when they are used more than once and the shorthand `$0` would be hard to tell apart.
- When a closure body grows past a few lines, extract a named function: it can be tested and read on its own.

6.5 Practice exercises

1. Write a function `repeatText(_ text: String, times: Int = 1) -> String` that returns the text repeated `times` times, separated by spaces, and call it with and without the second argument.
2. What is the printed output of the following code?

Swift

```swift
let values = [5, 3, 9, 1]
print(values.sorted())
print(values.sorted(by: >))
print(values)
```

3. Rewrite this loop as a single expression with `map` and `join`.

Swift

```swift
var result = ""
for number in 1...3 {
    result += "\\(number * number)"
    if number != 3 {
        result += "-"
    }
}
print(result)
```

4. Explain the difference between a closure capturing a `var` local and a closure capturing a value stored in a `let` constant.
5. Write an `@escaping` closure parameter and a caller that supplies a closure which prints after the function returns.

Solutions

1. One solution:

Swift

```swift
func repeatText(_ text: String, times: Int = 1) -> String {
    return Array(repeating: text, count: times).joined(separator: " ")
}
print(repeatText("hi"))
print(repeatText("hi", times: 3))
```

output

```
hi
hi hi hi
```

2. The output is `[1, 3, 5, 9]`, then `[9, 5, 3, 1]`, then `[5, 3, 9, 1]`: `sorted` returns a new array and leaves the original untouched.
3. `print((1...3).map { $0 * $0 }.joined(separator: "-"))`, which prints `1-4-9`.
4. A `var` local captured by a closure is a shared box, so the closure and the enclosing scope observe the same value and later changes are visible. A value stored in a `let` constant is copied into the closure, so the closure has a value of its own.
5. One solution:

Swift

```swift
var pending: [() -> Void] = []
func later(_ body: @escaping () -> Void) {
    pending.append(body)
}
later { print("executed later") }
for body in pending { body() }
```

output

```
executed later
```

6.6 Mini test

1. What separates the argument label from the parameter name in a declaration? A) A colon B) A comma C) Two parameter names in a row, with no separator word D) `->`
2. How is a variadic parameter's set of values represented in the body? A) As a tuple B) As an array C) As a set D) As a dictionary
3. Which operator is used to pass a value to an `inout` parameter? A) `&` B) `*` C) `ref` D) `inout`
4. What is the type of the function `func compute(_ a: Int, _ b: Int) -> Double`? A) `(Int, Int) -> Double` B) `Int -> Double` C) `(Int, Double) -> Int` D) `Double`
5. What does `sorted(by: >)` do to its array? A) Sorts it in place B) Returns a new array and leaves the original unchanged C) Reverses it D) Removes duplicates
6. True or false: a closure written as the last argument of a call can be placed after the parentheses.
7. What does `map` return? A) The same collection B) A new collection with transformed elements C) A single value D) Nothing
8. Which keyword marks a closure parameter that may outlive the call? A) `@autoclosure` B) `@escaping` C) `@discardableResult` D) `@inout`
9. What is the printed result of `print([1, 2, 3].reduce(0) { $0 + $1 })`?
10. What happens when a function without `@discardableResult` has an unused result?

Answer key and explanations

1. C - `func greet(person name: String)` names the label first and the parameter second, with no word between them.
2. B - the variadic values arrive as an array, so `values.reduce(0, +)` works on them.
3. A - the argument is passed with `&`, and it must be a mutable `var`.

4. A - the function type lists the parameter types and then the result.
5. B - `sorted` is not a mutating method; `sort()` is the mutating form.
6. True - that is the trailing closure syntax, and it is the usual style.
7. B - `map` transforms element by element and returns a new collection.
8. B - `@escaping` marks the closure so the compiler knows the call may finish before the closure runs.
9. `6` - the reduction starts at `0` and adds each element.
10. The compiler emits a warning that the result is unused.

Chapter 7 - Structures, classes and enumerations

7.1 Summary

- Structures and enumerations are value types, and classes are reference types, which is the first question to ask when designing a new type.
- Value types are copied on assignment, so a change to one copy never affects the others; reference types are shared, so every reference sees the same object.
- A structure automatically receives a memberwise initialiser from its stored properties, plus an implicit `init()` when every stored property has a default.
- A method on a value type that changes a stored property must be marked `mutating`.
- Properties come in several forms: stored, lazy, computed, and property observers (`willSet` and `didSet`) that watch a store.
- Subscripts add bracket access with any number of parameters and are the usual way to give a custom type index-based access.
- Classes support inheritance, single class only, with `override` required when a member replaces a parent's member and `final` to forbid further overriding.
- Class instances have identity, compared with `===`, and are managed by automatic reference counting, so strong cycles must be broken with `weak` or `unowned`.
- Initialisation is a two-phase process: every stored property must have a value before the instance can be used, and a subclass must call a designated initialiser of its superclass.
- Enumerations are first-class types with raw values or associated values, methods, computed properties and `CaseIterable`, and are value types.

7.2 Concepts

Value types versus reference types

output

```
struct Point { var x = 0 }            final class Screen { var width = 0 }

var a = Point(x: 1)                   let s1 = Screen(width: 100)
var b = a                             let s2 = s1
b.x = 9                               s2.width = 200

a.x == 1   (copy)                     s1.width == 200  (shared)
```

Question	Value type (`struct`, `enum`)	Reference type (`class`)
Assignment	copies	shares
Identity	none	`===` compares
Thread safety	safe to share	needs synchronisation
Inheritance	not available	available
Deinitialiser	not available	available
Usual role	data, values, states	entities, engine, services

Value semantics make reasoning local: a function that receives a struct cannot change the caller's value. Reference types are needed when several parts of a program must look at the same object, for example a window, a connection or a model that drives the screen.

Properties

Form	Meaning
Stored	a value kept in the instance, `var position = 0`
Constant	a stored property that never changes, `let name = "Ada"`
Lazy	computed once when first accessed, `lazy var report = build()`
Computed	derived from other properties, no storage of its own
Type	belongs to the type, `static var count = 0`
Observer	`willSet`/`didSet` run code when a stored property changes

A computed property has a `get` block and, when the property is `var`, a `set` block that receives `newValue` unless it names another parameter. Observers are only available on stored properties (including inherited ones), and a `lazy` property cannot have observers because nothing happens before its first access.

Property wrappers, such as `@State`, `@Binding` and `@AppStorage`, are a compile-time feature that turns a property into a value with attached behaviour; the standard library provides a small set and UI frameworks provide more. Chapter 12 uses `@State` for exactly this reason.

Methods and subscripts

A method is a function that belongs to a type. Value types mark mutating methods with `mutating`; classes do not, because their properties are already mutable through shared references. Type methods are written `static` and are called on the type itself, with `class func`

instead when a class wants subclasses to override them.

Subscripts look like indexing with parameters:

output

```
grid[row, column]       subscript(row: Int, column: Int) -> Int
rows[2...4]             a range subscript over a collection
matrix[1, 2] = 30       a subscript with both get and set
```

Inheritance

Swift supports single inheritance: a class has one direct superclass. A subclass inherits stored properties, methods and subscripts, overrides an inherited member with the `override` keyword, and can call the parent implementation with `super.`. Marking a class or a member `final` forbids further overriding, which is both a design statement and a compiler hint.

output

```
class Vehicle
     |
     +-- class Car
     |        |
     |        +-- final class ElectricCar
     |
     +-- class Truck
```

Initialisation

Initialisation is two-phase: first every stored property of the instance is given a value, then the instance can be used and its members called. Two kinds of initialiser exist in classes:

Kind	Role
Designated	sets every stored property, calls a designated initialiser of the superclass
Convenience	calls another initialiser of the same class, must end up at a designated one

Other forms worth knowing: a failable initialiser `init?` returns `nil` when the construction is impossible, a `required init` is required in every subclass, and a `deinit` runs when the last reference to an instance disappears. Structures and enumerations have initialisers but no inheritance and no `deinit`.

Automatic reference counting

Swift counts strong references to each class instance and frees the instance when the count reaches zero. A strong reference cycle, for example a parent that keeps an array of children while each child keeps a strong reference back to the parent, prevents that count from ever reaching zero.

Tool	Rules
`weak var x: T?`	zeroing reference, optional, use when the owner may disappear first

Tool	Rules
`unowned var x: T`	non-optional reference that must outlive the holder, crashes when accessed after deallocation
Strong capture in a closure	add a capture list, `[weak self]` or `[unowned self]`

Choosing a type

Need	Choice
A value with a few fields	`struct`
A set of related cases	`enum`
Shared state, identity, a lifecycle	`class`
An interface without implementation	`protocol`
A type that works with many element types	generic

Swift's own standard library is built around value types: `Array`, `Dictionary`, `Set` and `String` are all structures.

7.3 Examples

Example 1: a structure with a memberwise initialiser and a mutating method.

Swift

```swift
struct Counter {
    var count = 0
    mutating func increment(by amount: Int = 1) {
        count += amount
    }
    func describe() -> String {
        return "count is \\(count)"
    }
}
var counter = Counter()
counter.increment()
counter.increment(by: 5)
print(counter.describe())
let frozen = Counter(count: 3)
print(frozen.describe())
```

output

```
count is 6
count is 3
```

The memberwise initialiser is generated from the stored properties, so `Counter(count: 3)` needs no explicit `init`. Calling a mutating method on a `let` constant is a compile-time error, which is why `counter` is a `var`.

Example 2: value types copy, reference types share.

Swift

```swift
struct Point {
    var x = 0
    var y = 0
}
var original = Point(x: 1, y: 2)
var copy = original
copy.x = 99
print(original.x, copy.x)

class Screen {
    var width: Int
    init(width: Int) {
        self.width = width
    }
}
let screenA = Screen(width: 1080)
let screenB = screenA
screenB.width = 2160
print(screenA.width, screenB.width)
print(screenA === screenB)
```

output

```
1 99
2160 2160
true
```

Copying is cheap and safe for value types because the storage is shared until one copy is changed, a technique called copy-on-write.

Example 3: computed properties, observers and lazy properties.

Swift

```swift
struct Temperature {
    var celsius: Double = 25.0
    var fahrenheit: Double {
        get {
            return celsius * 9 / 5 + 32
        }
        set {
            celsius = (newValue - 32) * 5 / 9
        }
    }
}
var temperature = Temperature()
print(temperature.fahrenheit)
temperature.fahrenheit = 212
print(temperature.celsius)

class StepCounter {
    var steps: Int = 0 {
        willSet {
            print("setting \\(newValue)")
        }
        didSet {
            print("was \\(oldValue), now \\(steps)")
        }
    }
}
let tracker = StepCounter()
tracker.steps = 1500

class Report {
    let name = "Weekly"
    lazy var title: String = "\\(name) report"
    func describe() -> String {
        return title
    }
}
let report = Report()
print(report.describe())
```

output

```
77.0
100.0
setting 1500
was 0, now 1500
Weekly report
```

`fahrenheit` is computed from `celsius` and back, and both directions work with a single stored property. The lazy property is built only when `title` is first read.

Example 4: type methods and subscripts.

Swift

```swift
struct TicketGenerator {
    static var serial = 1000
    static func next() -> Int {
        defer { serial += 1 }
        return serial
    }
}
print(TicketGenerator.next())
print(TicketGenerator.next())

struct Grid {
    var cells: [[Int]]
    subscript(row: Int, column: Int) -> Int {
        get {
            return cells[row][column]
        }
        set {
            cells[row][column] = newValue
        }
    }
}
var grid = Grid(cells: [[1, 2], [3, 4]])
print(grid[0, 1])
grid[1, 0] = 30
print(grid.cells)
```

output

```
1000
1001
2
[[1, 2], [30, 4]]
```

The static state belongs to the type, so both calls draw from the same serial number without passing a generator around.

Example 5: enumerations with raw values, methods and associated values.

Swift

```swift
enum Suit: String, CaseIterable {
    case hearts, diamonds, clubs, spades
    var isRed: Bool {
        return self == .hearts || self == .diamonds
    }
}
print(Suit.hearts.rawValue)
print(Suit.diamonds.isRed, Suit.clubs.isRed)
print(Suit.allCases.count)
print(Suit(rawValue: "clubs") as Any)
print(Suit(rawValue: "jokers") as Any)

enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    var area: Double {
        switch self {
        case .circle(let radius):
            return Double.pi * radius * radius
        case .rectangle(let width, let height):
            return width * height
        }
    }
    func describe() -> String {
        switch self {
        case .circle(let radius):
            return "circle r=\\(radius)"
        case .rectangle(let width, let height):
            return "rectangle \\(width)x\\(height)"
        }
    }
}
print(Shape.circle(radius: 1).area)
print(Shape.rectangle(width: 3, height: 2).area)
print(Shape.rectangle(width: 3, height: 2).describe())
```

output

```
hearts
true false
4
Optional(Suit.clubs)
nil
3.141592653589793
6.0
rectangle 3x2
```

Raw values give every case a stable value of one type, and switching over a computed property of `self` is the idiomatic way to derive behaviour from the case.

Example 6: inheritance, designated initialisers and overriding.

Swift

```swift
class Vehicle {
    let name: String
    init(name: String) {
        self.name = name
    }
    func describe() -> String {
        return "vehicle: \\(name)"
    }
}
final class Bicycle: Vehicle {
    var gears: Int
    init(gears: Int) {
        self.gears = gears
        super.init(name: "bicycle")
    }
    override func describe() -> String {
        return "\\(super.describe()) with \\(gears) gears"
    }
}
let bicycle = Bicycle(gears: 2)
print(bicycle.describe())
print(bicycle.name, bicycle.gears)
```

output

```
vehicle: bicycle with 2 gears
bicycle 2
```

The designated initialiser sets `gears`, then calls the parent's designated initialiser, after which the parent can be used. `override` is required and `super.describe()` reuses the parent's work.

Example 7: reference cycles, `weak` and `unowned`.

Swift

```swift
class Customer {
    let name: String
    var card: CreditCard?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("customer gone")
    }
}
class CreditCard {
    let number: String
    unowned var owner: Customer
    init(number: String, owner: Customer) {
        self.number = number
        self.owner = owner
    }
    deinit {
        print("card gone")
    }
}
func issueCard() {
    var customer: Customer? = Customer(name: "Ada")
    if let owner = customer {
        owner.card = CreditCard(number: "1234", owner: owner)
        if let card = owner.card {
            print("card number \\(card.number)")
        }
    }
    print("dropping the customer")
    customer = nil
}
issueCard()
```

output

```
card number 1234
dropping the customer
customer gone
card gone
```

The customer strongly owns the card, so the card's reference back must be weak or unowned; `unowned` is right when the card cannot outlive its owner. The deinitialiser of the customer runs before its stored properties are released.

Example 8: nested data with an enumeration.

Swift

```
enum Json {
    case number(Double)
    case text(String)
    case list([Json])
    func describe() -> String {
        switch self {
        case .number(let value):
            return "\\(value)"
        case .text(let value):
            return "\"\\(value)\""
        case .list(let items):
            return "[" + items.map { $0.describe() }.joined(separator: ", ") + "]"
        }
    }
}
let document: Json = .list([.number(1), .text("two"), .list([.number(3)])])
print(document.describe())

indirect enum Tree {
    case leaf
    case node(left: Tree, right: Tree)
    func depth() -> Int {
        switch self {
        case .leaf:
            return 1
        case .node(let left, let right):
            return 1 + max(left.depth(), right.depth())
        }
    }
}
print(Tree.node(left: .node(left: .leaf, right: .leaf), right: .leaf).depth())
```

output

```
[1, "two", [3]]
3
```

`indirect` lets a case store the enumeration itself, which is how a recursive tree type is written.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using a class for plain data	shared mutable state appears where none was intended	start with a struct, move to a class when identity or sharing matters
Forgetting `mutating` on a method that changes a struct	the compiler rejects the method	add `mutating`, or make the method return a new value
Writing `=` to compare two class instances	it compares identity, not content	implement `Equatable` when value equality is meant
Two objects strongly referencing each other	the instances are never freed and memory grows	make one reference `weak` or `unowned`
Using `unowned` for a reference that may disappear	a crash when the reference is used	use `weak` and an optional

Mistake	Why it hurts	Best practice
Forgetting `override` in a subclass	the compiler rejects the method, and a typo silently creates a new one	add `override`, keep overrides small
Nested deeply chained convenience initialisers	the reader must follow every call to find the real setup	keep one or two designated initialisers at the top
Giving every class `init` complexity	a subclass cannot know what to call	prefer a few full designated initialisers
Storing mutable state in a type that should be a value	changes leak to distant parts of the program	use value types, and share through a single reference type when needed

- Prefer structures and enumerations for model types; they are cheap to copy, safe to pass across boundaries, and the standard library agrees with you.
- When a class is required, look at each reference it keeps and ask whether it should be `weak`; that question keeps a large graph honest.

7.5 Practice exercises

1. Write a struct `Vector` with `x` and `y`, a `mutating func add(other: Vector)` and a computed property `length`.
2. Why does the compiler reject `let counter = Counter(); counter.increment()` when `increment()` is marked `mutating`?
3. Which of these types are value types: `struct Book`, `class Library`, `enum Genre`, `enum Genre: String`, `class Reader`?
4. Given `class Node { var next: Node?; deinit { print("gone") } }`, construct a cycle between two nodes, then rewrite the class so the cycle is broken with `weak`.
5. Write an enumeration `Media` with a case `photo(width: Int, height: Int)` and a computed property `pixels`, then print the pixels for a 1920 by 1080 photo.

Solutions

1. One solution:

Swift

```swift
struct Vector {
    var x: Double
    var y: Double
    mutating func add(other: Vector) {
        x += other.x
        y += other.y
    }
    var length: Double {
        return (x * x + y * y).squareRoot()
    }
}
var v = Vector(x: 3, y: 4)
v.add(other: Vector(x: 9, y: -4))
print(v.x, v.y, v.length)
```

output

```
12.0 0.0 12.0
```

2. Because a constant cannot be modified: `mutating` methods change `self`, so the binding must be a `var`.
3. `struct Book`, `enum Genre` and `enum Genre: String`. `class Library` and `class Reader` are reference types.
4. The cycle is `a.next = b` and `b.next = a`. Making the link weak, `weak var next: Node?`, keeps the reference optional and lets the nodes disappear when nothing else points at them.
5. One solution:

Swift

```swift
enum Media {
    case photo(width: Int, height: Int)
    var pixels: Int {
        switch self {
        case .photo(let width, let height):
            return width * height
        }
    }
}
print(Media.photo(width: 1920, height: 1080).pixels)
```

output

```
2073600
```

7.6 Mini test

1. Which types have value semantics in Swift? A) classes B) structures and enumerations C) protocols D) closures
2. What does a structure automatically receive from its stored properties? A) A `deinit` B) A memberwise initialiser C) A reflection API D) Nothing
3. Which keyword must a struct method use to change a stored property? A) `mutating` B) `override` C) `inout` D) `var`

4. How is a computed property different from a stored property? A) It is cached on first access B) It is derived from other properties each time it is read C) It cannot be read D) It is only available on classes
5. Which operator compares two class instances for identity? A) `==` B) `===` C) `~=` D) `is`
6. True or false: a class in Swift can inherit from more than one class.
7. What must a designated initialiser of a subclass do before it finishes? A) Call a convenience initialiser B) Call a designated initialiser of its superclass C) Nothing D) Call `deinit`
8. When does `deinit` run? A) Every time a property changes B) When the last strong reference to the instance is removed C) When the program starts D) Never
9. What is the difference between `weak` and `unowned`? A) There is none B) `weak` is an optional that becomes `nil`, while `unowned` is a non-optional reference that must outlive the holder C) `unowned` is an optional, `weak` is not D) `weak` only works with protocols
10. What does `case .photo(let width, let height)` bind in a `switch` over a `Media` value?

Answer key and explanations

1. B - structures and enumerations are copied on assignment, classes are shared.
2. B - the memberwise initialiser is generated unless an explicit `init` is written in the same scope.
3. A - mutating methods change `self`, so the binding must be a variable.
4. B - the value is derived, not stored; `lazy` gives the caching behaviour instead.
5. B - `===` tests identity, and `!==` negates it.
6. False - a class has a single direct superclass; multiple capabilities are expressed with protocols.
7. B - a designated initialiser must delegate up the chain so the parent can set its own properties.
8. B - automatic reference counting removes the instance when its strong reference count reaches zero.
9. B - `weak` zeroes the optional and can be tested, `unowned` assumes the object is still alive.
10. The two associated values of the case, as local constants usable inside the case's body.